

Санкт-Петербургский Национальный Исследовательский Университет
ИТМО

Факультет Программной Инженерии и Компьютерной Техники



Лабораторная работа № 4

По дисциплине

База данных

Вариант №8913

Выполнил студент группы Р3132:

Васидов Мухаммадсаид Абдуфаттохович

Преподаватель:

Гаврилов Антон Валерьевич

Санкт-Петербург – 2026

Содержание

Текст задания.....	3
Запрос 1	4
Индексы	5
EXPLAIN ANALYZE.....	5
Запрос 2	6
Индексы	7
EXPLAIN ANALYZE.....	7
Вывод.....	8

Текст задания

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.ИД.

Фильтры (AND):

а) Н_ЛЮДИ.ФАМИЛИЯ < Соколов.

б) Н_СЕССИЯ.ЧЛВК_ИД < 126631.

Вид соединения: INNER JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_ВЕДОМОСТИ.ИД, Н_СЕССИЯ.ИД.

Фильтры (AND):

а) Н_ЛЮДИ.ФАМИЛИЯ > Соколов.

б) Н_ВЕДОМОСТИ.ДАТА < 1998-01-05.

с) Н_СЕССИЯ.ИД > 27640.

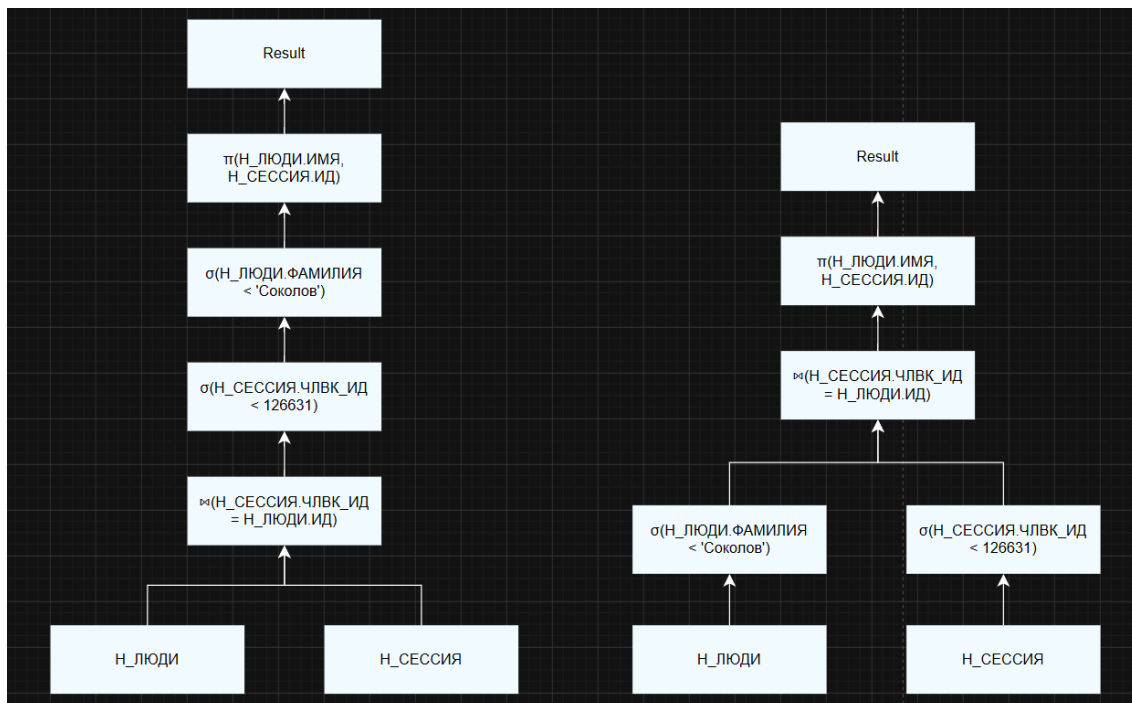
Вид соединения: INNER JOIN.

Запрос 1

```
SELECT Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.ИД
FROM Н_ЛЮДИ
  INNER JOIN Н_СЕССИЯ
    ON Н_СЕССИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
WHERE Н_ЛЮДИ.ФАМИЛИЯ < 'Соколов'
  AND Н_СЕССИЯ.ЧЛВК_ИД < 126631;
```

Результат:

	ИМЯ character varying (15)	ИД integer
1	Вячеслав	1976
2	Ильдар	1977
3	Вячеслав	1980
4	Ильдар	1981
5	Вячеслав	1984
6	Ильдар	1985
7	Антон	1353
8	Александр	1356



Оптимальным является план №2, так как фильтр `ФАМИЛИЯ < 'Соколов'` оставляет примерно половину строк `Н_ЛЮДИ`, и строить хэш-таблицу из меньшей выборки выгоднее. Условие `ЧЛВК_ИД < 126631` дополнительно сокращает число строк `Н_СЕССИЯ`.

Индексы

```
CREATE INDEX "ИНД_ЛЮДИ_ФАМИЛИЯ" ON "Н_ЛЮДИ" USING BTREE("ФАМИЛИЯ");  
CREATE INDEX "ИНД_СЕС_ЧЛВК_ИД" ON "Н_СЕССИЯ" USING BTREE("ЧЛВК_ИД");
```

При добавлении индексов планы выполнения запросов изменятся: вместо полного сканирования таблиц будет производиться Index Scan, а оптимизатор, скорее всего, перейдёт на Nested Loop Join, так как для каждой строки из Н_ЛЮДИ можно быстро найти совпадения в Н_СЕССИЯ по индексу

EXPLAIN ANALYZE

Nested Loop (cost=0.29..275.18 rows=2014 width=17) (actual time=0.028..2.368 rows=2048 loops=1)

-> Seq Scan on "Н_СЕССИЯ" (cost=0.00..117.90 rows=2974 width=8) (actual time=0.011..0.745 rows=2974 loops=1)

Filter: ("ЧЛВК_ИД" < 126631)

Rows Removed by Filter: 778

-> Memoize (cost=0.29..0.47 rows=1 width=17) (actual time=0.000..0.000 rows=1 loops=2974)

Cache Key: "Н_СЕССИЯ"."ЧЛВК_ИД"

Cache Mode: logical

Hits: 2841 Misses: 133 Evictions: 0 Overflows: 0 Memory Usage: 15kB

-> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..0.46 rows=1 width=17) (actual time=0.003..0.003 rows=1 loops=133)

Index Cond: ("ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД")

Filter: (("ФАМИЛИЯ")::text < 'Соколов'::text)

Rows Removed by Filter: 0

"Planning Time: 0.364 ms"

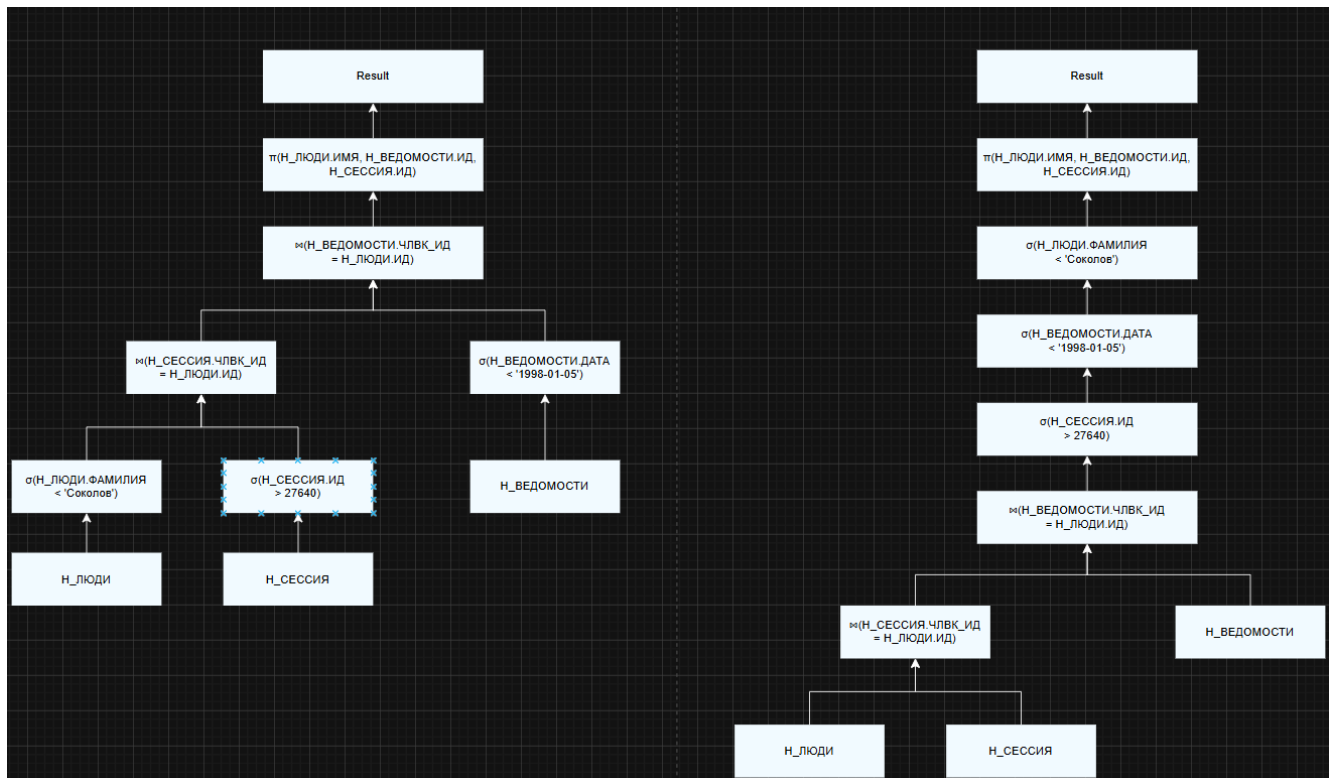
"Execution Time: 2.500 ms"

Запрос 2

```
SELECT Н_ЛЮДИ.ИМЯ,  
       Н_ВЕДОМОСТИ.ИД,  
       Н_СЕССИЯ.ИД  
FROM Н_ЛЮДИ  
  INNER JOIN Н_СЕССИЯ  
    ON Н_СЕССИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД  
  INNER JOIN Н_ВЕДОМОСТИ  
    ON Н_ВЕДОМОСТИ.ЧЛВК_ИД = Н_ЛЮДИ.ИД  
WHERE Н_ЛЮДИ.ФАМИЛИЯ > 'Соколов'  
      AND Н_ВЕДОМОСТИ.ДАТА < '1998-01-05'  
      AND Н_СЕССИЯ.ИД > 27640;
```

Результат:

ИМЯ	ИД	ИД
character varying (15)	integer	integer



Оптимальным является план №1. Фильтр $\text{Н_ВЕДОМОСТИ.ДАТА} < \text{'1998-01-05'}$ обрабатывается быстро, так как ведомости за первые дни 1998 года не существует. Соединение по цепочке FK ($\text{Н_ЛЮДИ} \rightarrow \text{Н_СЕССИЯ} \rightarrow \text{Н_ВЕДОМОСТИ}$) уменьшает промежуточный результат на каждом шаге и снижает суммарные затраты памяти.

Индексы

```
CREATE INDEX "ИНД_ЛЮДИ_ФАМИЛИЯ" ON "Н_ЛЮДИ" USING BTREE("ФАМИЛИЯ");
CREATE INDEX "ИНД_ВЕД_ДАТА" ON "Н_ВЕДОМОСТИ" USING BTREE("ДАТА");
CREATE INDEX "ИНД_СЕС_ИД" ON "Н_СЕССИЯ" USING BTREE("ИД");
CREATE INDEX "ИНД_СЕС_ЧЛВК_ИД" ON "Н_СЕССИЯ" USING BTREE("ЧЛВК_ИД");
CREATE INDEX "ИНД_СЕС_ВЕД_ИД" ON "Н_СЕССИЯ" USING BTREE("ВЕД_ИД");
```

Все условия фильтрации используют операторы сравнения (>, <), поэтому оптимален BTREE-индекс, поддерживающий диапазонный поиск. При добавлении индексов оптимизатор перейдёт на Nested Loop + Index Scan, полный просмотр таблиц будет исключён.

EXPLAIN ANALYZE

Nested Loop (cost=0.86..16.44 rows=1 width=21) (actual time=0.005..0.006 rows=0 loops=1)

-> Nested Loop (cost=0.58..15.50 rows=1 width=25) (actual time=0.004..0.005 rows=0 loops=1)

-> Index Scan using "ВЕД_ДАТА_I" on "Н_ВЕДОМОСТИ" (cost=0.29..7.19 rows=1 width=8) (actual time=0.004..0.004 rows=0 loops=1)

Index Cond: ("ДАТА" < '1998-01-05 00:00:00'::timestamp without time zone)

-> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..8.30 rows=1 width=17) (never executed)

Index Cond: ("ИД" = "Н_ВЕДОМОСТИ"."ЧЛВК_ИД")

Filter: (("ФАМИЛИЯ")::text > 'Соколов'::text)

-> Index Scan using "SYS_C003500_IFK" on "Н_СЕССИЯ" (cost=0.28..0.92 rows=1 width=8) (never executed)

Index Cond: ("ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")

Filter: ("ИД" > 27640)

Planning Time: 1.156 ms

Execution Time: 0.046 ms

Вывод

При выполнении лабораторной работы я ознакомился с составлением SQL-запросов с соединением таблиц (INNER JOIN), использованием диапазонных фильтров, а также с методами анализа и оптимизации планов выполнения запросов в PostgreSQL.